BLG 562E: Parallel Computing For GPUs using CUDA

Spring 2026

Week 6- Performance Tools and Debugging

Section Outline

- Performance Tools
  - NVVP, nvprof, Nsigth system, Nsight compute
- Debugging
  - cuda-gdb
  - cuda-memcheck

https://developer.nvidia.com/performance-analysis-tools

Performance Analysis Tools

NVIDIA Nsight Systems

NVIDIA® Nsight™ Systems is a system-wide performance analysis tool designed to visualize application's algorithm, help you select the largest opportunities to optimize, and tune to scale efficiently across any quantity of CPUs and GPUs in your computer; from laptops to DGX servers.

NVIDIA® Nsight™

The ultimate development platform for heterogeneous computing. Work with powerful debugging and profiling tools, optimize the performance of your CPU and GPU code. Find out about the Eclipse Edition and the graphics debugging enabled Visual Studio Edition.

NVIDIA Visual Profiler

This is a cross-platform performance profiling tool that delivers developers vital feedback for optimizing CUDA C/C++ applications. First introduced in 2008, Visual Profiler supports all CUDA capable NVIDIA GPUs shipped since 2006 on Linux, Mac OS X, and Windows.

TAU Performance System®

This is a profiling and tracing toolkit for performance analysis of hybrid parallel programs written in CUDA, and pyCUDA., and OpenACC.

VampirTrace

A performance monitor which comes with CUDA, and PyCUDA support to give detailed insight into the runtime behavior of accelerators. Enables extensive performance analysis and optimization of hybrid programs.

The PAPI CUDA Component

A hardware performance counter measurement technology for the NVIDIA CUDA platform which provides access to the hardware counters inside the GPU. Provides detailed performance counter information regarding the execution of GPU kernels.

The NVIDIA CUDA Profiling Tools Interface

(CUPTI) provides performance analysis tools with detailed information about GPU usage in a system. CUPTI is used by performance analysis tools such as the NVIDIA Visual Profiler, TAU and Vampir Trace.

NVIDIA Topology-Aware GPU Selection

(NVTAGS) is a toolset for HPC applications that enables faster solve times with high GPU communication-to-application run-time ratios. NVTAGS intelligently and automatically assigns GPUs to message passing interface (MPI) processes, thereby reducing overall GPU-to-GPU communication time.

Performance Tools

Performance Opportunities

- Application level opportunities
  - Overall GPU utilization and efficiency
  - Memory copy efficiency
- Kernel level opportunities
  - Instruction and memory latency hiding/ reduction
  - Efficient use of memory bandwidth
  - Efficient use of compute resources

CUDA Kernel Optimization

- Trace the application
- Identify the hot spot and profile it
  - Total time, number of calls
- Identify the performance limiter
  - Memory Bandwidth
  - Instruction Throughput
  - Latency
- Optimize the code
- Iterate

What Limits Kernel Performance?

- Memory bandwidth
  - Low number of operations per byte
- Instruction bandwidth
  - High number of operations per byte
- Latency
  - Unable to fill memory or arithmetic pipelines
  - First two things to check:
    - Occupancy
    - Memory accesses (coalesced/uncoalesced accesses)
  - Other things to check (if needed):
    - Control flow efficiency (branching, idle threads because of synchronization)
    - Divergence
    - Bank conflicts in shared memory

How do we determine what is limiting a given kernel?

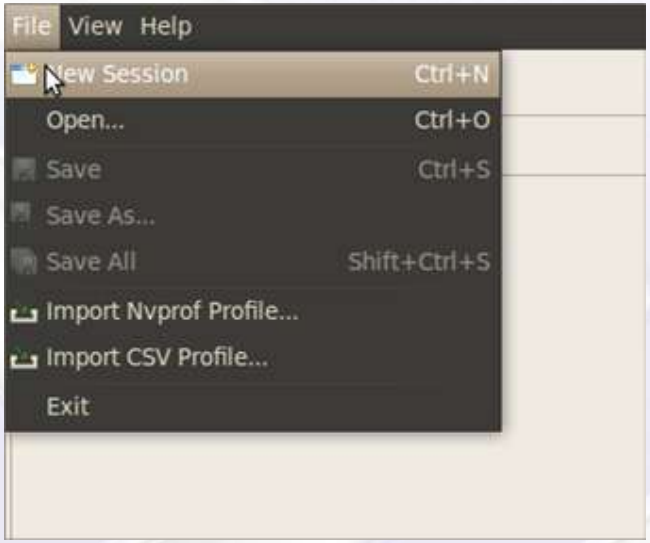
- Use NVIDIA Profiling Tools
  - NVIDIA® Visual Profiler
    - Standalone (NVVP)
    - Integrated into NVIDIA® NsightTM Eclipse Edition (nsight)
  - nvprof
    - Command-line profiler
  - NVIDIA® NsightTM Visual Studio Edition

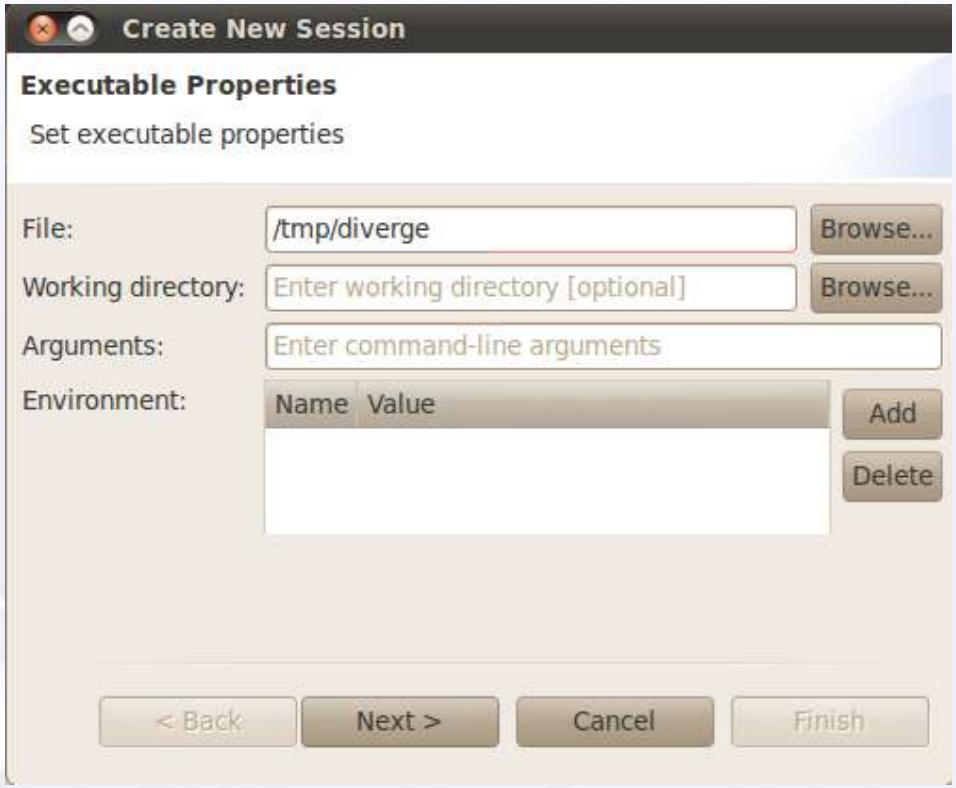
- Cuda Profiling Tools Interface (CUPTI)
  - Enables creation of profiling and tracing tools that target CUDA applications
  - Designed for tools vendors, not CUDA developers

Nvidia Visual Profiler (NVVP)

- Unified Memory Profiling
- Multi-Process Service (MPS) Profiling
- Detailed Kernel Profile
- Shared Memory Source Level Analysis
- Mix of Instructions for a Kernel
- Inefficient SM Utilization Detection
- Remote Profiling

NVVP – Profiling Session





NVVP -Timeline

File View Run Help

*diverge.vp

0.05 s | 0.055 s | 0.06 s | 0.065 s

Process: 25290
Thread: -1813960928
Runtime API — cudaMemcpyAsync
Driver API
[0] GeForce GTX 480
Context 1 (CUDA)
MemCpy (HtoD)
MemCpy (DtoH)
Compute — VecThe... Vec50(in... Vec1o... Vec1of32x(int*, int*, int*, int)
56.3% [4] Vec1of32x(in... — Vec1of32x(int*, int*, int*, int)
10.6% [4] Vec1of32(int... — Vec1o...
13.3% [4] Vec50(int*, i... — Vec50(in...
12.5% [4] VecThen(int*... — VecThe...
7.3% [4] Vec32of32(int...
0.0% [4] VecEmpty(void)
Streams
Stream 1 — VecThe... Vec50(in... Vec1o... Vec1of32x(int*, int*, int*, int)

Properties | Detail Graphs

Vec32of32(int*, int*, int*, int)

Name	Value
Start	70.642 ms
End	72.064 ms
Duration	1.422 ms
Grid Size	[256,1,1]
Block Size	[256,1,1]
Registers/Thread	11
Shared Memory/Block	0 bytes
Occupancy	
Theoretical	100%
L1 Cache Configuration	
Shared Memory Requested	48 KB
Shared Memory Executed	48 KB

Analysis | Details | Console | Settings

Session diverge.vp

Executable

File: bin/diverge [Browse...]
Working directory: Enter working directory [optional] [Browse...]
Arguments: Enter command-line arguments
Environment: Name | Value

Profiling Flow

- Understand CPU behavior on timeline
  - Add profiling “annotations” to application
- NVIDIA Tools Extension (NVTX)
  - Custom markers and time ranges
  - Custom naming
- Focus profiling on region of interest
  - Reduce volume of profile data
  - Improve usability of Visual Profiler
  - Improve accuracy of analysis
- Analyze for optimization opportunities

Annotations: NVIDIA Tools Extension

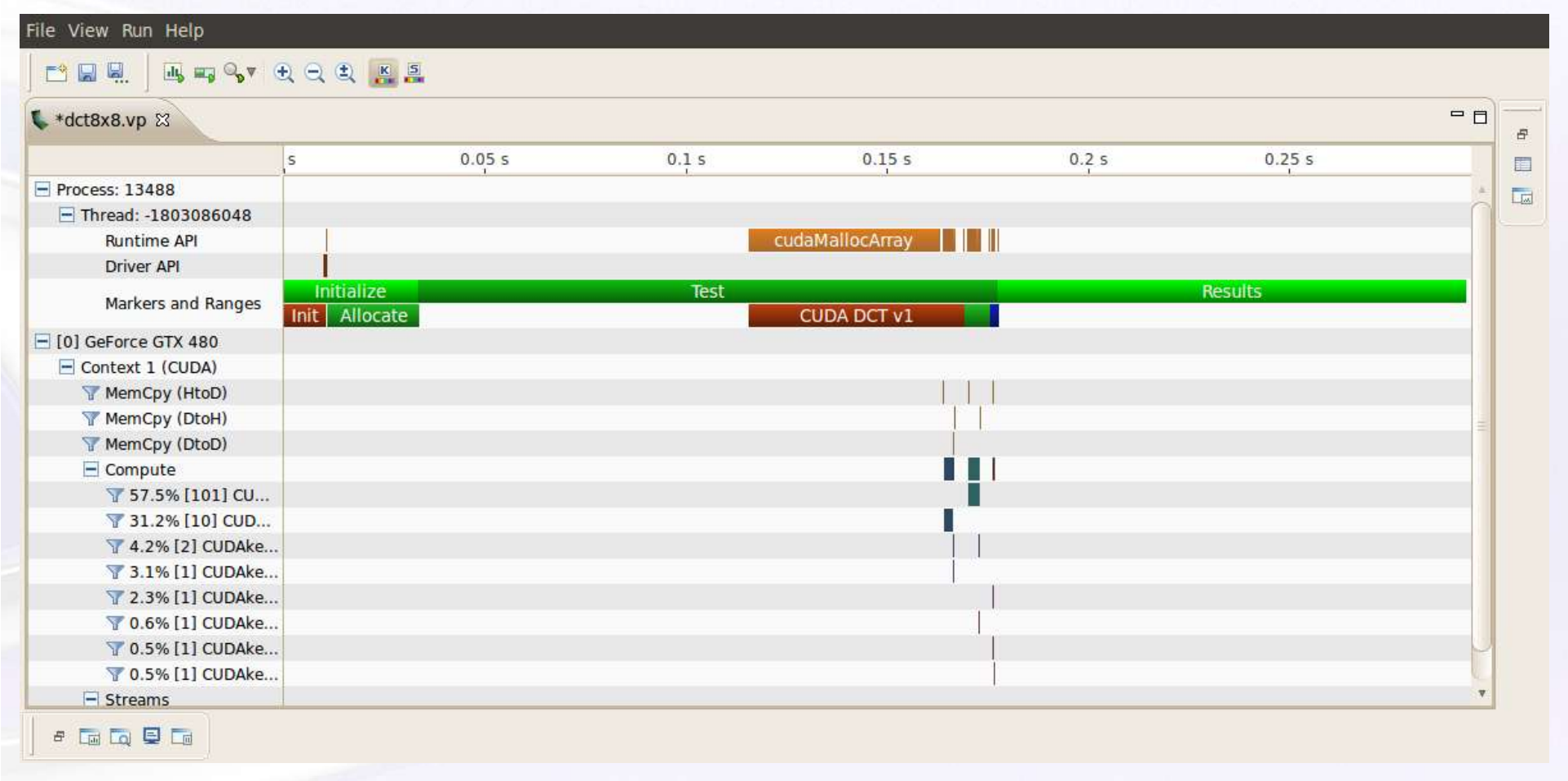
- Developer API for CPU code
- Installed with CUDA Toolkit (libnvToolsExt.so)
- Time Ranges and Markers
  - Range: nvtxRangeStart(), nvtxRangeEnd()
  - Instantaneous marker: nvtxMark()

Example: Time Ranges

- Use time ranges API to mark initialization, test, and results

```
…
nvtxRangeId_t id0 = nvtxRangeStart("Initialize");
< init code >
nvtxRangeEnd(id0);
nvtxRangeId_t id1 = nvtxRangeStart("Test");
< compute code >
nvtxRangeEnd(id1);
…
```

Example: Time Ranges



Profile Region Of Interest

- cudaProfilerStart() / cudaProfilerStop() in CPU code
- Specify representative subset of app execution
  - Manual exploration and analysis simplified
  - Automated analysis focused on performance critical code

```
for (i = 0; i < N; i++) {
    if (i == 12) cudaProfilerStart();
    <loop body>
    if (i == 15) cudaProfilerStop();
}
```

Enable Region of Interest

- Insert cudaProfilerStart() / cudaProfilerStop()
- Disable profiling at start of application

Analysis

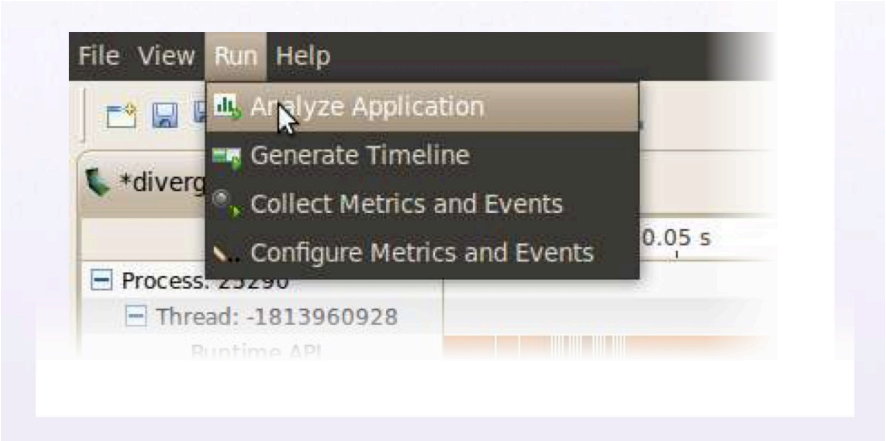
- Visual inspection of timeline
- Automated Analysis
- Metrics and Events

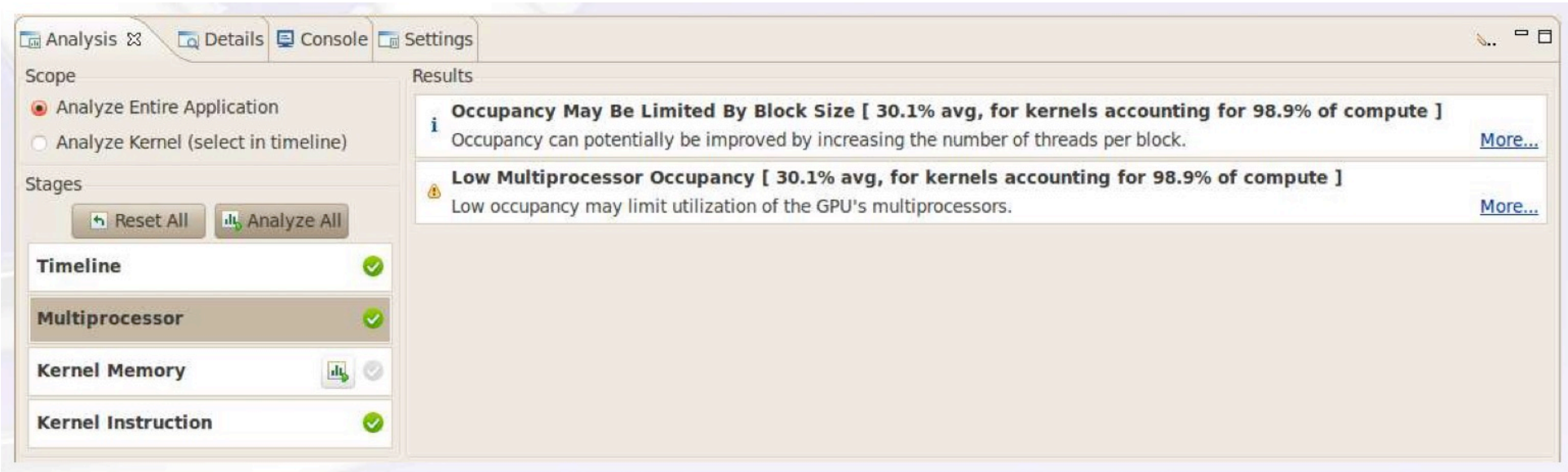
Visual Inspection

- Understand CPU/GPU interactions
  - Use nvToolsExt to mark time ranges on CPU
  - Is application taking advantage of both CPU and GPU?
  - Is CPU waiting on GPU? Is GPU waiting on CPU?
- Look for potential concurrency opportunities
  - Overlap memcpy and kernel
  - Concurrent kernels
- Automated analysis does some of this

Automated Analysis – Application

- Analyze entire application
  - Timeline
  - Hardware performance counters





Analysis Documentation

Low Memcpy Throughput [997.19 MB/s avg, for memcpys accounting for 68.1% of all memcpy time]

The memory copies are not fully using the available host to device bandwidth. More...

Search: Go Scope: All topics

Content

- Visual Profiler Optimizatic
  - Preface
  - Parallel Computing with CU
  - Performance Metrics
  - Memory Optimizations
    - Data Transfer Between H
      - Pinned Memory
      - Asynchronous Transfe
      - Zero Copy
    - Device Memory Spaces
      - Allocation
  - Execution Configuration Op
  - Instruction Optimizations
  - Control Flow
  - Recommendations and Bes

Between Host and Device

Pinned Memory

Page-locked or pinned memory transfers attain the highest bandwidth between the host and the device. On PCIe ×16 Gen2 cards, for example, pinned memory can attain greater than 5 GBps transfer rates.

Pinned memory is allocated using the `cudaMallocHost()` or `cudaHostAlloc()` functions in the Runtime API. The `bandwidthTest.cu` program in the CUDA SDK shows how to use these functions as well as how to measure memory transfer performance.

Pinned memory should not be overused. Excessive use can reduce overall system performance because pinned memory is a scarce resource. How much is too much is difficult to tell in advance, so as with all optimizations, test the applications and the systems they run on for optimal performance parameters.

Parent topic: Data Transfer Between Host and Device

Copyright © 2011 NVIDIA Corporation | www.nvidia.com

NVIDIA

Analysis Properties

- Highlight a kernel or memcpy in timeline
  - Properties shows analysis results for that specific kernel / memcpy
  - Optimization opportunities are flagged

Properties | Detail Graphs

CUDAkernel2DCT(float*, float*, int)

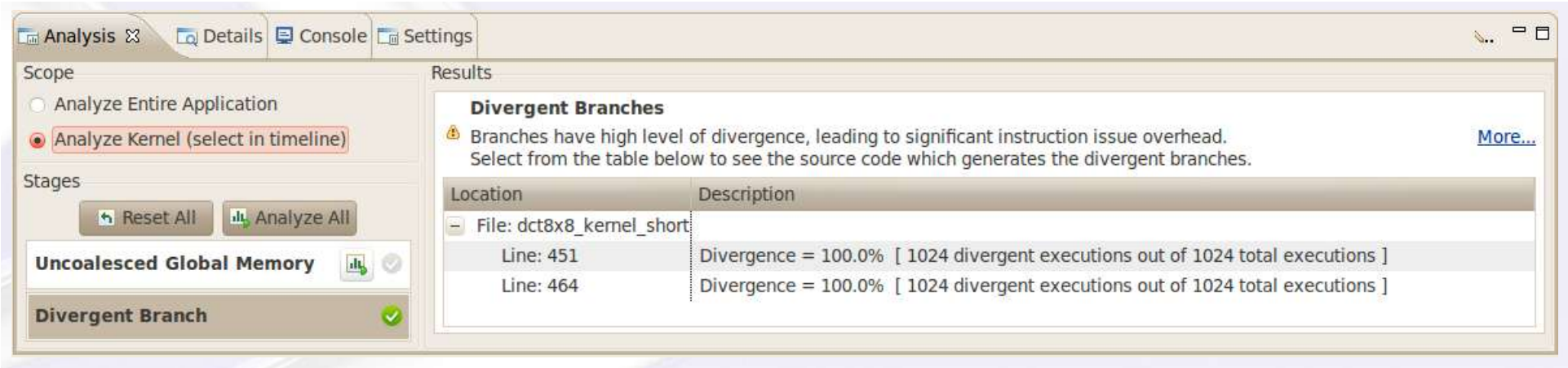
Name	Value
Duration	21.117 µs
Grid Size	[16,32,1]
Block Size	[8,4,2]
Registers/Thread	35
Shared Memory/Block	2.062 KB
Memory	
Global Load Efficiency	100%
Global Store Efficiency	100%
Instruction	
Branch Divergence Overhead	0%
Occupancy	
Achieved	29.4%
Theoretical	33.3%
Limiter	Block Size
L1 Cache Configuration	
Shared Memory Requested	48 KB
Shared Memory Executed	48 KB

Uncoalesced Global Memory Accesses

- Access pattern determines number of memory transactions
  - Report loads/stores where access pattern is inefficient

Divergent Branches

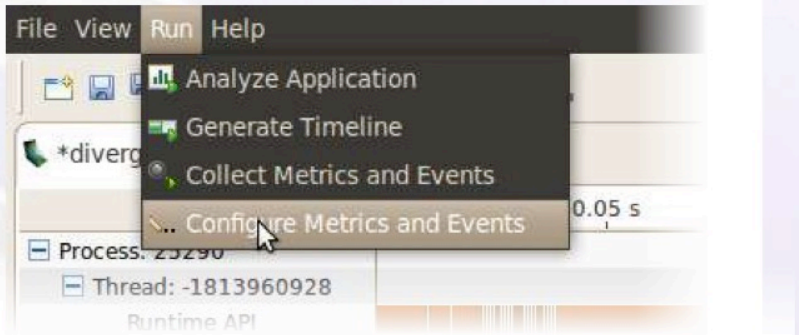
- Divergent control-flow for threads within a warp
  - Report branches that have high average divergence

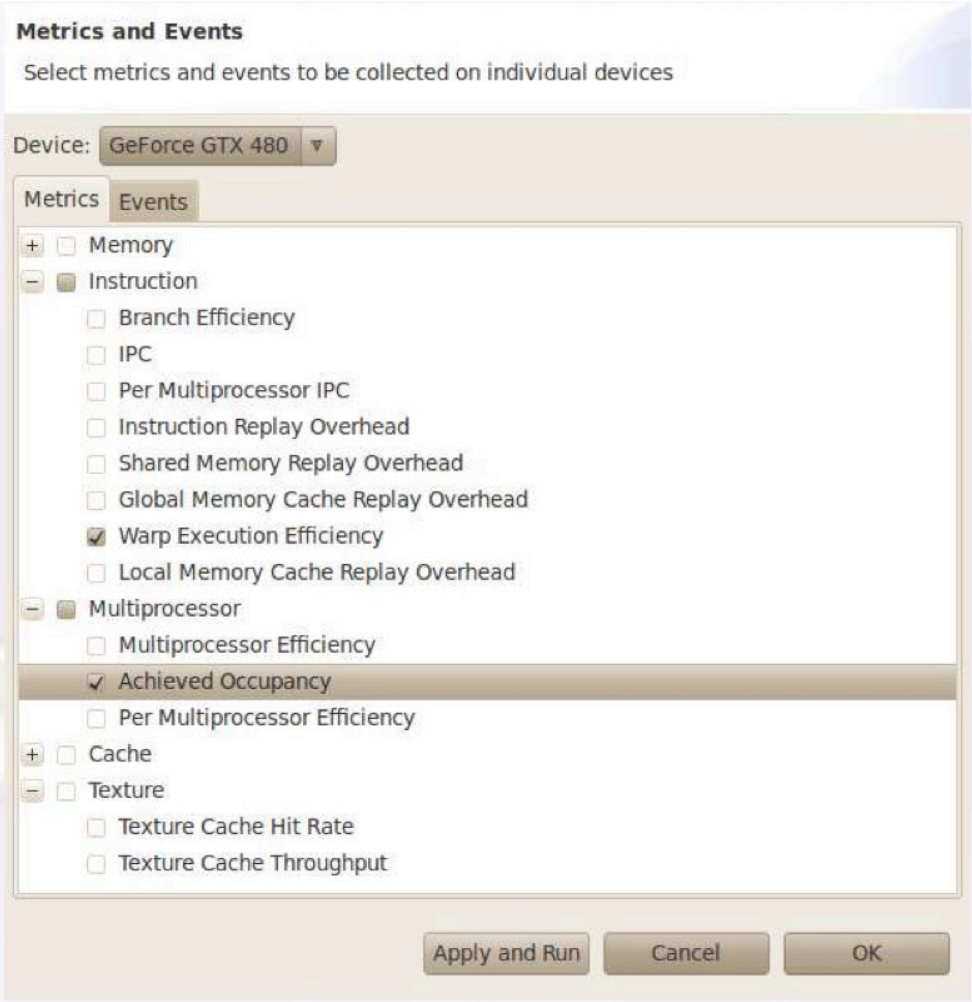


Enabling Source Correlation

- Source correlation requires that source/line information be embedded in executable
  - Available in debug executables: nvcc –G
  - New flag for optimized executables: nvcc -lineinfo

Metrics and Events





Command-line profiler: nvprof

- Textual reports
  - Summary of GPU and CPU activity
  - Trace of GPU and CPU activity
  - Event collection
- Use “nvprof” to collect data
  - Visualize timeline with Visual Profiler

nvprof usage

- nvprof [nvprof_args] <app> [app_args]

- Argument help
  - nvprof --help

nvprof – GPU Summary

> nvprof dct8x8

```
======== Profiling result:
 Time(%)      Time   Calls       Avg       Min       Max  Name
   49.52    9.36ms     101   92.68us   92.31us   94.31us  CUDAkernel2DCT(float*, float*, int)
   37.47    7.08ms      10  708.31us  707.99us  708.50us  CUDAkernel1DCT(float*,int, int,int)
    3.75  708.42us       1  708.42us  708.42us  708.42us  CUDAkernel1IDCT(float*,int,int,int)
    1.84  347.99us       2  173.99us  173.59us  174.40us  CUDAkernelQuantizationFloat()
    1.75  331.37us       2  165.69us  165.67us  165.70us  [CUDA memcpy DtoH]
    1.41  266.70us       2  133.35us   89.70us  177.00us  [CUDA memcpy HtoD]
    1.00  189.64us       1  189.64us  189.64us  189.64us  CUDAkernelShortDCT(short*, int)
    0.94  176.87us       1  176.87us  176.87us  176.87us  [CUDA memcpy HtoA]
    0.92  174.16us       1  174.16us  174.16us  174.16us  CUDAkernelShortIDCT(short*, int)
    0.76  143.31us       1  143.31us  143.31us  143.31us  CUDAkernelQuantizationShort(short*)
    0.52   97.75us       1   97.75us   97.75us   97.75us  CUDAkernel2IDCT(float*, float*)
    0.12   22.59us       1   22.59us   22.59us   22.59us  [CUDA memcpy DtoA]
```

nvrof – GPU Summary (csv)

> nvprof --csv dct8x8

```
======== Profiling result:
Time(%),Time,Calls,Avg,Min,Max,Name
,ms,,us,us,us,
49.51,9.35808,101,92.65400,92.38200,94.19000,"CUDAkernel2DCT(float*, float*, int)"
37.47,7.08288,10,708.2870,707.9360,708.7070,"CUDAkernel1DCT(float*, int, int, int)"
3.75,0.70847,1,708.4710,708.4710,708.4710,"CUDAkernel1IDCT(float*, int, int, int)"
1.84,0.34802,2,174.0090,173.8130,174.2060,"CUDAkernelQuantizationFloat(float*, int)"
1.75,0.33137,2,165.6850,165.6690,165.7020,"[CUDA memcpy DtoH]"
1.42,0.26759,2,133.7970,89.89100,177.7030,"[CUDA memcpy HtoD]"
1.00,0.18874,1,188.7360,188.7360,188.7360,"CUDAkernelShortDCT(short*, int)"
0.94,0.17687,1,176.8690,176.8690,176.8690,"[CUDA memcpy HtoA]"
0.93,0.17594,1,175.9390,175.9390,175.9390,"CUDAkernelShortIDCT(short*, int)"
0.76,0.14281,1,142.8130,142.8130,142.8130,"CUDAkernelQuantizationShort(short*, int)"
0.52,0.09758,1,97.57800,97.57800,97.57800,"CUDAkernel2IDCT(float*, float*, int)"
0.12,0.02259,1,22.59300,22.59300,22.59300,"[CUDA memcpy DtoA]"
```

nvprof – GPU Trace

> nvprof --print-gpu-trace dct8x8

```
======== Profiling result:
     Start  Duration  Grid Size  Block Size  Regs  SSMem  DSMem  Size    Throughput  Name
  167.82ms  176.84us  -          -           -     -      -      1.05MB  5.93GB/s    [CUDA memcpy HtoA]
  168.00ms  708.51us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  168.95ms  708.51us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  169.74ms  708.26us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  170.53ms  707.89us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  171.32ms  708.12us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  172.11ms  708.05us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  172.89ms  708.38us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  173.68ms  708.31us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  174.47ms  708.15us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  175.26ms  707.95us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  176.05ms  173.87us  (64 64 1)  (8 8 1)     27    0B     0B     -       -           CUDAkernelQuantization (…)
  176.23ms   22.82us  -          -           -     -      -      1.05MB  45.96GB/s   [CUDA memcpy DtoA]
```

nvprof – CPU/GPU Trace

> nvprof --print-gpu-trace --print-api-trace dct8x8

```
======== Profiling result:
     Start  Duration  Grid Size  Block Size  Regs  SSMem  DSMem  Size    Throughput  Name
  167.82ms  176.84us  -          -           -     -      -      1.05MB  5.93GB/s    [CUDA memcpy HtoA]
  167.81ms    2.00us  -          -           -     -      -      -       -           cudaSetupArgument
  167.81ms   38.00us  -          -           -     -      -      -       -           cudaLaunch
  167.85ms    1.00ms  -          -           -     -      -      -       -           cudaDeviceSynchronize
  168.00ms  708.51us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
  168.86ms    2.00us  -          -           -     -      -      -       -           cudaConfigureCall
  168.86ms    1.00us  -          -           -     -      -      -       -           cudaSetupArgument
  168.86ms    1.00us  -          -           -     -      -      -       -           cudaSetupArgument
  168.86ms    1.00us  -          -           -     -      -      -       -           cudaSetupArgument
  168.87ms       0ns  -          -           -     -      -      -       -           cudaSetupArgument
  168.87ms   24.00us  -          -           -     -      -      -       -           cudaLaunch
  168.89ms  761.00us  -          -           -     -      -      -       -           cudaDeviceSynchronize
  168.95ms  708.51us  (64 64 1)  (8 8 1)     28    512B   0B     -       -           CUDAkernel1DCT(float*, …)
```

nvprof – Event Query

> nvprof --devices 0 --query-events

```
======== Available Events:
                                Name   Description
Device 0:
         Domain domain_a:
                  sm_cta_launched:  Number of thread blocks launched on a multiprocessor.

               l1_local_load_hit:  Number of cache lines that hit in L1 cache for local
memory load accesses. In case of perfect coalescing this increments by 1, 2, and 4 for 32, 64
and 128 bit accesses by a warp respectively.

              l1_local_load_miss:   Number of cache lines that miss in L1 cache for local
memory load accesses. In case of perfect coalescing this increments by 1, 2, and 4 for 32, 64
and 128 bit accesses by a warp respectively.

              l1_local_store_hit:  Number of cache lines that hit in L1 cache for local
memory store accesses. In case of perfect coalescing this increments by 1, 2, and 4 for 32,
64 and 128 bit accesses by a warp respectively.
```

nvprof – Event Collection

> nvprof --devices 0 --events branch,divergent_branch

```
======== Profiling result:
                    Invocations       Avg       Min       Max  Event Name
Device 0
         Kernel: CUDAkernel1IDCT(float*, int, int, int)
                              1    475136    475136    475136  branch
                              1         0         0         0  divergent_branch
         Kernel: CUDAkernelQuantizationFloat(float*, int)
                              2    180809    180440    181178  branch
                              2      6065      6024      6106  divergent_branch
         Kernel: CUDAkernel1DCT(float*, int, int, int)
                             10    475136    475136    475136  branch
                             10         0         0         0  divergent_branch
         Kernel: CUDAkernelShortIDCT(short*, int)
                              1    186368    186368    186368  branch
                              1      2048      2048      2048  divergent_branch
         Kernel: CUDAkernel2IDCT(float*, float*, int)
                              1     61440     61440     61440  branch
                              1         0         0         0  divergent_branch
```

nvprof – Profile Data Export/Import

- Produce profile into a file using –o
  - > nvprof –o profile.out <app> <app args>
- Import into Visual Profiler
  - File menu -> Import nvprof Profile...
- Import into nvprof to generate textual outputs
  - > nvprof –i profile.out
  - > nvprof –i profile.out --print-gpu-trace
  - > nvprof –i profile.out --print-api-trace

Nvidia Nsight Compute

- https://developer.nvidia.com/nsight-compute
- https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html#command-line-option
- https://www.youtube.com/watch?v=DnwZ6ZTLw50&t=138s
- https://docs.nvidia.com/nsight-visual-studio-edition/4.6/Content/Analysis/Report/CudaExperiments/KernelLevel/PerformanceCounters.htm
- Nvidia's official kernel profiling guide:

https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html

- A hands-on Nsight training exercise:

https://github.com/NVIDIA/nsight-training/tree/master/cuda/2020_gtc

Debugging

Best Practices for Debugging

- Determine scope of the bug
  - Incorrect result
  - Unspecified Launch Failure (ULF)
  - Crash
  - Hang
  - Slow execution
- Reproduce with a debug build
  - Compile your app with –g –G
  - Rerun
- Performance Issues
  - Use the visual profiler
- Correctness Issues
  - First cuda-memcheck stand-alone
  - Then cuda-gdb if needed
  - “printf”

Debugging Using “printf”

- “printf” is supported on devices of sm_20 and higher
- Requires inclusion of “stdio.h”
- Use with caution
  - Fixed buffer size
  - Flushed under certain circumstances
    - E.g next time a kernel launched
  - Not flushed by default at the end of an application
    - Must be forced by cudaDeviceReset()

“cuda-gdb”

- All the standard GDB debugging features supported
- Seamless CPU and GPU debugging within a single session
- Supports multiple GPUs, multiple contexts, multiple kernels
- Source and Assembly (SASS) Level Debugging
- Runtime Error Detection (stack overflow,...)

Debugging using “cuda-gdb”

- Need to compile with –g –G options
- Use –gencode options
  - Nvcc –g –G –gencode arc=compute_35 code=sm_35 …
- Run with
  - >cuda-gdb myapp

(cuda-gdb) _

Debugging using “cuda-gdb”

- Using breakpoints
  - By name
    - (cuda-gdb) break my_kernel
      (cuda-gdb) break _Z6kernelIfiEvPT_PT0
  - By file name and line number
    - (cuda-gdb) break myapp.cu:185
  - At every kernel launch
    - (cuda-gdb) set cuda break_on_launch application
  - By address
    - (cuda-gdb) break *0x3e840a8
- CUDA application at a breakpoint == Frozen display
- Multiple Solutions:
  - Console mode: no X server
  - Multiple GPUs: one for display, one for compute
  - Remote Debugging: SSH, VNC, ...

Auto-stepping

- The “autostep” command
  - Define a window of instructions where we think the problem occurs
  - cuda-gdb will single-step all the instructions within that window automatically and without user intervention

```
>(cuda-gdb) autostep foo.cu:25 for 20 lines
```

Focus

- Many threads to deal with
  - How to decide which one the user wants?
- Concept of current focus
  - Either host thread or device thread
  - Which kernel/block/thread the user is looking at
  - cuda-gdb commands apply to the current focus
- Two different views for a device thread
  - Hardware coordinates: device, SM, warp, lane
  - Software coordinates: kernel, block, thread
  - Mapping between the two sets of coordinates

Mapping Between Software and Hardware Coordinates

- A device may execute multiple kernels
- A block may run on multiple warps

- Thread Focus
  - Some commands apply only to the thread in focus
    - Print local or shared variables
    - Print registers
    - Print stack contents

Focus Querying

- Query commands
  - cuda <list of coordinates>
  - thread
- If focus set to device thread
  - > (cuda-gdb) cuda kernel block thread

    kernel 1, block (0, 0, 0), thread (0, 0, 0)
  - > (cuda-gdb) cuda device kernel block warp thread

    kernel 1, block (0, 0, 0), thread (0, 0 ,0), device 0,warp 0
- If focus set to host thread
  - > (cuda-gdb) thread

    [Current thread is 1 ...]
  - > (cuda-gdb) cuda thread

    Focus not set on any active CUDA kernel

Focus Switch

- Switch command
  - cuda <list of coordinate-value pairs>
  - thread <host thread id>
- Only switch the specified coordinates
  - Current coordinates are assumed in case of non-specified coordinates
  - If no current coordinates, best effort to match request
  - Error if cannot match request

Focus Switch

- > (cuda-gdb) cuda kernel 1 block 1 thread 2,0
  - [Switching focus to CUDA kernel 1, grid 2, block (1, 0, 0), thread (2, 0, 0), device 0, sm 5, warp 0, lane 2]
- > (cuda-gdb) cuda block (1, 0, 0) lane 7 sm 5
  - [Switching focus to CUDA kernel 1, grid 2, block (1, 0, 0), thread (7, 0, 0), device 0, sm 5, warp 0, lane 7]
- > (cuda-gdb) cuda kernel 1
  - [Switching focus to CUDA kernel 1, grid 2, block (0, 0, 0), thread (0, 0, 0), device 0, sm 1, warp 0, lane 0]
- > (cuda-gdb) cuda thread 256
  - Request cannot be satisfied. CUDA focus unchanged

Program State Inspection

- (cuda-gdb) info cuda <topic>
- Topics include:
  - Kernels
  - Blocks
  - Threads
  - Devices
  - SMs
  - Warps
  - Lanes

To get info on Devices

- To obtain the list of devices in the system:
  - (cuda-gdb) info cuda devices
- To obtain the list of running kernels:
  - (cuda-gdb) info cuda kernels
- To obtain the list of running threads for kernel 2:
  - (cuda-gdb) info cuda threads kernel 2
    - Threads are displayed in (block,thread) ranges
    - Divergent threads are in separate ranges
- Stack Trace:
  - (cuda-gdb) info stack

“cuda-memcheck”

- Stand-alone run-time error checker tool
- Detects memory errors like stack overflow,…
- Same spirit as valgrind
- No need to recompile the application
- Not all the error reports are precise
- Once used within cuda-gdb, the kernel launches are blocking

cuda-memcheck Errors

- Illegal global address
- Misaligned global address
- Stack memory limit exceeded
- Illegal shared/local address
- Misaligned shared/local address
- Instruction accessed wrong memory
- PC set to illegal value
- Illegal instruction encountered
- Illegal global address

Using “cuda-memcheck”

- Integrated in cuda-gdb
  - More precise errors when used from cuda-gdb
  - Must be activated before the application is launched

(cuda-gdb) set cuda memcheck on

Source

- NVIDIA Corp.